

上海财经大学《程序设计基础》上机考试卷(A)

(2024 至 2025 学年 第 1 学期)

答题及答案提交要求:

- (1) 请务必在 D 盘上创建一个答题目录, 例如: `d:\project`, 将提供的程序文件拷贝到该目录, 利用该目录答题, 以免机器死机导致数据丢失。
- (2) 答题完成准备提交答案时, 请在 D 盘上创建一个**答案文件夹**, 该文件夹的名字用你的学号+姓名命名。如学号为 2024000123, 姓名为“李明”, 需要创建一个目录“2024000123 李明”。**将你做的答案保存在 D 盘你的答题文件夹下。**
- (3) **对于题目 1-4**, 请将你完成后的 `main.cpp` 文件名分别修改为 `one.cpp`, `two.cpp`, `three.cpp`, `four.cpp`, 保存在 D 盘你的答题文件夹下。
- (4) **对于题目 5**, 请将完成后的 `mystring.cpp` 保存在 D 盘你的答题文件夹下。
- (5) 考试结束时, **请提交 D 盘你的答题文件夹, 在和监考老师确认提交成功后才可离开。**

注意事项:

- (1) 程序的基本框架已经给出, 你仅需要补充部分缺少的代码;
- (2) 试题可能没留出足够空行, 应根据需要确定代码长度。

1、(函数、循环控制 20 分)

以下程序用于输入一组数据, 并对数据进行统计。系统首先 (1) 请用户输入一个整数 n ($1 \leq n \leq 100$), 表示输入数据的个数; (2) 然后请用户连续输入 n 个整数, 存储到一个数组 `arr` 中, 并同时统计数组中正数、负数和零的个数, 以及所有正数的和; (3) 逆序打印输入的数据; (4) 打印统计结果。

请补充完成该程序中缺少的代码。

运行示例:

```
Please input the number of data: 6
Please input 6 integers:
-10 0 12 -8 15 4
The elements in reverse order:
4 15 -8 12 0 -10
# of Positive Numbers:3
# of Negative Numbers:2
# of Zeros 1
Average of Positive Numbers:10.3333
```

```
#include <iostream>
using namespace std;

int main() {

    int n;
    cout << "Please input the number of data: ";
    cin >> n;
```

```

int arr[100]; // 定义数组
int positive = 0, negative = 0, zero = 0; // 用于统计正数、负数、零的个数
int posSum = 0; // 统计正数的和

cout << "Please input " << n << " integers: " << endl;
// 从键盘读取 n 个整数，存储在数组 arr[] 中，
// 并统计正数、负数、零的个数，以及正数的和，
// 请补充此处缺少的代码①

    cout << "The elements in reverse order: " << endl;
    // 逆序打印数组的内容，元素之间用空格隔开
    // 请补充此处缺少的代码②

// 输出统计结果
cout << "# of Positive Numbers:" << positive << endl;
cout << "# of Negative Numbers:" << negative << endl;
cout << "# of Zeros " << zero << endl;

// 计算并输出正数的平均值
if (positive > 0) {
    double average = (double) posSum / positive; // 保留浮点数类型
    cout << "Average of Positive Numbers:" << average << endl;
} else {
    cout << " Average of Positive Numbers:0" << endl;
}

return 0;
}

```

2、(字符串 20 分)

编写程序从屏幕读入一串字符串，将其中的大小写字母进行如下处理：

- (1) 将大写字母改为小写字母；
- (2) 将小写字母改为大写字母；

输出处理后的字符串。

样例输入：

Hello, World! 123

样例输出：

hELLO, wORLD! 123

请完成以下程序：

```

#include <iostream>
using namespace std;

```

```

void caseFlip(char str[]);

int main() {
    char st[100];
    cout << "please input a string:" << endl;
    cin.getline(st, 100);

    caseFlip(st);

    cout << st << endl;
    return 0;
}

// 函数 caseFlip 将字符串 str 中的大小写字母进行转换，其他字符不变
void caseFlip(char str[])
{
    // 请补充缺少的代码①

}

```

3. (指针与动态数组 20 分)

某学校需要一个程序统计各个班级学生的成绩信息，要求编写的程序能够根据班级人数不同，动态申请存储学生成绩的数组。在 main 函数中，首先创建一个动态数组，以便存储当前班级的学生分数，然后调用函数 inputScores，要求用户从键盘输入当前班级学生的成绩，最后统计输出班级的平均成绩（average，保留两位小数）以及方差(variance)。

注：方差计算公式为 $\sigma^2 = \sum (x - \mu)^2 / N$ 其中： σ^2 表示方差， $\sum$ 表示求和， x 表示各个数据， μ 表示数据的均值。

程序运行示例：

```

Please input the number of students: 5
Please input 5 scores for the class:
40 50 60 70 80
The average score:60.00
The variance of the scores:200.00

```

```

#include <iostream>
#include <iomanip>
using namespace std;

void inputScores(int ScoreArr[], int classSize);
void printStatistics(int ScoreArr[], int classSize);

// 主函数
int main() {
    int *scoreArray;    // 指向为当前班级分配的动态数组（用于存储成绩）
}

```

```

int classSize; // 班级人数

cout << "Please input the number of students: ";
cin >> classSize;

// 创建动态数组,用于存储当前班级的成绩, 请补充缺少的代码 ①
scoreArray = ① ;
if(!scoreArray)
{
    cout << "memory allocation failed!" ;
    exit(0);
}

// 输入成绩
inputScores(scoreArray, classSize);

// 计算并输出平均成绩、方差
printStatistics(scoreArray, classSize);

// 释放动态分配的内存, 请补充此处缺少的代码 ②

return 0;
}

void inputScores(int ScoreArr[], int classSize) {
    //接受数组 ScoreArr 以及 classSize 作为参数, 从键盘输入当前班级学生的成绩
    cout << "Please input "<< classSize << " scores for the class:"<< endl;
    for (int i = 0; i < classSize; i++){
        cin >> ScoreArr[i];
    }
}

void printStatistics(int ScoreArr[], int classSize)
{
    // 计算平均成绩, 请补充缺少的代码 ③

    // 计算方差, 请补充缺少的代码 ④

    // 输出平均成绩、方差
    cout << "The average score:"

```

```

        << fixed << setprecision(2) << average << endl;

    cout << "The variance of the scores:"
        << fixed << setprecision(2) << variance << endl;

}

```

4. (递归, 20 分)

小青蛙正在爬楼梯，它每次可以选择跳 1 格、2 格或者 4 格。请编写一个程序，输入楼梯的总格数 n ($1 \leq n \leq 30$)，计算小青蛙共有多少种不同的跳法。请补充完成缺少的代码，要求使用递归实现。

样例输入：

10

样例输出：

169

提示：递归问题的关键是求得递推式。 本题求不同的跳法数，可考虑原问题是否可分为几种情况（每种情况是规模更小的问题），如果能够求得几个更小问题的跳法数，那么将它们相加即可得到总的跳法数。

运行示例：

```

please input the total number of stairs:10
the total number of ways:169

```

```

#include <iostream>
using namespace std;
int countWays(int n);

// 主函数， 注意：请不要修改本函数
int main() {
    int n;
    cout << "please input the total number of stairs:";
    cin >> n;

    if (n < 1 || n > 30) {
        cout<<"Please input number between 1 and 30 !" <<endl;
        return 1;
    }

    int ways = countWays(n);
    cout << "the total number of ways:" << ways << endl;
    return 0;
}

// 递归函数计算跳法数
int countWays(int n) {
    // 请补充缺少的代码 ①

}

```

5. (类的实现, 20 分)

试定义一个字符串类 `MyString`, 实现其中的 (1) 重载赋值运算符 (`operator=`)、相等运算符 (`operator==`)、流插入 (`operator<<`) 运算符; (2) 重载 `operator+` 以便能够使用 `+` 运算符来连接两个 `MyString` 对象; (3) 实现成员函数 `find`, 该函数在字符串 `str` 中查找子串 `substr`, 返回子串在 `str` 中第一次出现的位置, 若 `str` 中不存在该子串返回 -1。

注意:

(1) 本程序包含 3 个文件: 头文件 `mystring.h`, 主程序文件 `main.cpp`, 类 `MyString` 的实现文件 `mystring.cpp` 只需要补充完成 `mystring.cpp` 中的函数即可, 注意: 这些函数的主要框架和部分代码已经提供, 仅需要补充其中缺少的代码。

(2) 你可以使用 `<cstring>` 库中的函数 `strlen`, `strcpy`, `strcat`, 这些函数的原型如下:

//功能: 返回字符串 `str` 中元素的个数, '`\0`' 不计算在内

//原型: `int strlen(const char *str);`

//功能: `strcpy` 函数用于将源字符串 `src` 复制到目标字符串 `dest` 中

// 原型: `char *strcpy(char *dest, const char *src);`

// 返回值: 返回目标字符串的指针。

// 注意: 源字符串必须以 '`\0`' 结束, 目标字符串必须有足够的空间来存放源字符串

//功能: `strcat` 函数用于将两个字符串拼接在一起。

//原型: `char *strcat(char *dest, const char *src);`

// 返回值: 返回目标字符串的指针。

//注意: 目标字符串必须有足够的空间来存放两个字符串拼接后的结果, 源字符串必须以 '`\0`' 结束。

程序运行示例:

```
str1 and str2 have the same content.
str3:World
str4:World
str5:World Hello
'He' at 6 of World Hello
```

(a) 头文件 `mystring.h`

// 注意: 请不要修改本文件

```
#include <iostream>
#include <cstring> // 你可以使用 strlen, strcpy, and strcat
using namespace std;
```

```
class MyString {
private:
    char* str;    // 指向 C 风格字符串的指针
public:
    // 构造函数
    MyString(const char* s = "") {
```

```

        str = new char[strlen(s) + 1];
        strcpy(str, s);
    }

    // 拷贝构造函数
    MyString(const MyString &source) ;

    // 析构函数
    ~MyString() { delete[] str; }

    // 重载赋值运算符
    MyString& operator=(const MyString &rhs) ;

    // 重载比较运算符
    friend bool operator==(const MyString &lhs, const MyString &rhs);

    // 重载+运算符， 连接两个 MyString 对象
    friend MyString operator+(const MyString &lhs, const MyString &rhs);

    // 重载+运算符， 连接一个 MyString 对象和一个 C 风格字符串
    friend MyString operator+(const MyString &lhs, const char* rhs);

    // 重载流插入运算符 <<
    friend ostream& operator<<(ostream &os, const MyString &source);

    // 返回 sub 在 str 中第一次出现的位置，如不存在该子串，返回-1
    int find(const char* sub) const;
};

```

(b) 主程序文件 main.cpp: 用于测试你编写的类

// 注意：请不要修改本文件（但在调试过程中你可以临时修改它）。

```

#include <iostream>
#include "mystring.h"
using namespace std;
int main() {

    MyString str1("Hello"), str2("Hello"), str3("World"); // 使用构造函数
    MyString str4(str3); // 使用拷贝构造函数
    MyString str5; // 使用缺省构造函数

    // 测试相等比较==
    if (str1 == str2)
        cout << "str1 and str2 have the same content.\n";
    else
        cout << "str1 and str2 do not have the same content.\n";

    cout <<"str3:" << str3 <<endl;    // 测试流插入运算符 <<
    cout <<"str4:" << str4 <<endl;    // 测试流插入运算符 <<

    str5 = str4+ " " + str1 ; // 测试两个重载的运算符 operator+
    cout <<"str5:" << str5 <<endl;    // 测试流插入运算符 <<
}

```

```

    cout << "'He' at " << str5.find("He") << " of " << str5; // 测试成员函数 find

    return 0;
}

```

(c) 类 MyString 的实现文件 mystring.cpp

// 请补充类 MyString 的实现程序中缺少的代码

//注：你可以使用库<cstring>中的 strlen, strcpy, and strcat 等函数

// MyString.cpp

```

#include <iostream>
#include "mystring.h"

```

// 拷贝构造函数

```

MyString::MyString(const MyString &source) {
    str=new char[strlen(source.str)+1];
    strcpy(str, source.str);
}

```

// 重载赋值运算符

```

MyString& MyString::operator=(const MyString &rhs) {
    // 请补充缺少的代码 ①

```

```

}

```

// 重载比较相等的运算符

```

bool operator==(const MyString &lhs, const MyString &rhs) {
    // 请补充缺少的代码 ②

```

```

}

```

// 重载流插入运算符

```

ostream& operator<<(ostream &os, const MyString &source) {
    // 请补充缺少的代码 ③

```

```

    return os;
}

```

// 重载 + 运算符用于连接两个 MyString 对象: MyString + MyString

```

MyString operator+(const MyString &lhs, const MyString &rhs) {
    MyString result; // 首先创建一个 MyString 对象，初始化为空字符串。
    // 请补充缺少的代码 ④ 提示：考虑如何根据 lhs, rhs 的值修改 result;

```

```

    return result;
}

```



```

// 重载 + 运算符用于连接一个 MyString 对象与一个 C 字符串连接: MyString + const char*
MyString operator+(const MyString &lhs, const char* rhs) {
    MyString result, rhs_string(rhs); // 创建一个新的 MyString 对象 rhs_string(用 rhs 初始化)
    result = lhs + rhs_string; // 调用前面定义的连接两个 MyString 对象的+ 运算符
    return result;
}

// find 函数找到子串 sub 在 MyString 对象中第一次出现的位置 (字符串下标值), 找不到返回-1
int MyString::find(const char* sub) const {
    if (sub == nullptr || *sub == '\0') {
        return 0; // Empty substring is always found at position 0
    }

    const char* mainStr = str;
    const char* subStr = sub;
    int mainLen = strlen(mainStr);
    int subLen = strlen(subStr);

    if (subLen > mainLen) {
        return -1; // Substring longer than main string, cannot be found
    }

    for (int i = 0; i <= mainLen - subLen; ++i) {
        // 请补充缺少的代码 (注: 可采用简单的字符串匹配) ⑤

    }

    return -1; // Substring not found
}

```